
Module 17 — GraphQL and WebSocket

Exercises

Spring Boot Edu



2026-09-24

Contents

How to Work	2
Exercise 1 — Authors and Their Books (Easy)	2
Exercise 2 — Placing an Order (Medium)	3
Exercise 3 — Live Stock Notifications (Medium)	3
Extra Challenge (Optional)	4

How to Work

1. The starter code is in `modules/17-graphql-websocket/exercise/`. Find the TODO comments.
2. The exercises need no Docker: the catalog and the inventory are in memory (package `catalog`, given).
3. Run the tests:

```
./mvnw -Pexercises -pl modules/17-graphql-websocket/exercise -am test
```

4. The exercise is done when all tests are green.
5. If you get stuck, read the hints first, then the solution in `modules/17-graphql-websocket/solution/`.

Exercise 1 — Authors and Their Books (Easy)

Goal: clients want a list of authors with their books. The schema does not have the field yet, and the books must be loaded without N+1.

Tasks:

- TODO 1a (`src/main/resources/graphql/schema.graphqls`) — give the type `Author` a field `books`: a non-null list of non-null `Books`.
- TODO 1b (`exercisel.AuthorController`) — implement `Query.authors` (all authors) and `Query.author(id)` (null for an unknown id).
- TODO 1c — implement `Author.books`. The books of all authors of a result must be loaded with **one** call to the catalog.

Hints:

- Lesson section 3.1: `[Book!]!`.
- Lesson section 3.2: `@QueryMapping` and `@Argument long id`. ID arrives as a string, and Spring converts it to long.
- Lesson section 3.3: `@BatchMapping Map<Author, List<Book>> books(List<Author> authors)` with `catalog.findBooksOf(Collection<Long>).Collectors.groupingBy(Book::authorId)` groups the books by author.
- Every author must be in the map, even one without books (`List.of()`).

Acceptance criteria: all 3 tests in `ExerciselTest` pass (one of them checks `catalog.bookQueries() == 1`).

Estimated time: 25 minutes

Exercise 2 — Placing an Order (Medium)

Goal: a `placeOrder` mutation. The schema and `OrderService` (validation, inventory) are given. The controller connects them to the schema and turns the service's exceptions into GraphQL errors.

Tasks (`exercise2.OrderController`):

- TODO 2a — `Mutation.placeOrder`: pass the input to `OrderService.place(...)`.
- TODO 2b — `Order.total`: `PlacedOrder` has no total, so compute the sum of unit price × quantity.
- TODO 2c — `OrderLine.book`: `PlacedLine` has only the ISBN; return the catalog book.
- TODO 2d — errors: `InvalidOrderException` and `OutOfStockException` → `BAD_REQUEST`, `UnknownBookException` → `NOT_FOUND`.

Hints:

- `@MutationMapping` with `@Argument OrderInput input`: the nested list of lines is bound to `OrderLineInput` records.
- The Java type name is not the schema type name here. Tell Spring the type: `@SchemaMapping(typeName = "Order")` and `@SchemaMapping(typeName = "Order-Line")`.
- A `BigDecimal` is written as a GraphQL `Float`.
- Lesson section 3.4: `@ExceptionHandler({InvalidOrderException.class, OutOfStockException.class})` for two exceptions, and `GraphQLErrorBuilder.newError(env).errorT`

Acceptance criteria: all 4 tests in `Exercise2Test` pass.

Estimated time: 30 minutes

Exercise 3 — Live Stock Notifications (Medium)

Goal: after every order, the shop's admin screen shows the new stock of the book at once. When only a few copies are left, a separate alert channel is notified.

The STOMP endpoint `/ws` and the broker for `/topic` are given (`WebSocketConfiguration`). The inventory publishes a Spring application event, `StockChanged(isbn, remaining)`, after every change.

Tasks (`exercise3.StockNotifier`):

- TODO 3a — on every `StockChanged` event, send a `StockLevel` to `/topic/stock/{isbn}` (for example `/topic/stock/9780321336781`).
- TODO 3b — when fewer than `LOW_STOCK` (3) copies are left, also send a `StockAlert` to `/topic/stock-alerts`.

Hints:

- An `@EventListener` method with a `StockChanged` parameter receives the events.
- Lesson section 3.6: `messaging.convertAndSend(destination, payload)`. The payload is converted to JSON.
- The test places orders with GraphQL (exercise 2 must be solved first) and listens with a STOMP client.

Acceptance criteria: both tests in `Exercise3Test` pass.

Estimated time: 20 minutes

Extra Challenge (Optional)

Add a GraphQL subscription `stockChanged(isbn: ID!): Int!` that delivers the same stock changes as exercise 3, but over GraphQL. Use a `Sinks.Many` as in lesson section 3.5, and write a test with `executeSubscription()` and `StepVerifier`.